

Spin Based Juggling with a Dual-Arm Humanoid

Anuriha Kodali Ily Rafaeli Taher Kapadia
MEAM 5170 University of Pennsylvania
Emails: {anuriha, ilyr, taherk}@seas.upenn.edu

Abstract—This project studies dynamic ball juggling using a simulated Unitree G1 humanoid robot with its hands replaced by rigid bowls. The goal is to repeatedly throw a ball from one bowl and catch it with the other, using only the arm joints for control. We work entirely in MuJoCo and explore two control strategies which share the same underlying sampling-based Model Predictive Control (MPC) infrastructure: (i) a direct MPC controller that optimizes a hand-crafted cost over future trajectories of the ball and bowls, and (ii) a phase-based “energy-shaping” controller that structures the throw into Spin, Tilt, and Release phases and shapes the kinetic energy of the ball through bowl motion. Both controllers rely on a predictive-sampling planner running at 500 Hz with a short horizon of 0.3 s. We describe the system modeling, cost design, phase logic, and a number of engineering techniques used to obtain physically plausible throws and catches, such as contact damping and analytic landing prediction. We qualitatively compare the behavior of the two controllers and highlight failure modes and successes, and how the system could be extended to more robust multi ball juggling and real hardware experiments.

I. INTRODUCTION

Juggling is a compact benchmark for dynamic manipulation; it requires fast motion, accurate timing, and tight coupling between ballistic flight and intermittent contact. Unlike quasi-static grasping, juggling pushes the robot to operate near its dynamic limits, making it a useful testbed for advanced control techniques such as model predictive control, policy search, and energy-based methods [2], [4].

In this project we consider a simplified juggling task using a simulated Unitree G1 humanoid robot [3], modeled in the MuJoCo physics engine [1]. Each arm has a bowl-shaped end-effector rigidly attached in place of the hand, and a single spherical ball is thrown back and forth between the two bowls. The task has several challenging aspects:

- **Hybrid dynamics:** hard contacts between ball and bowl, with sensitive dependence on impact timing.
- **Short flight times:** the ball remains close to the robot, so the arms must re-orient quickly in order to catch.
- **Underactuated ball motion:** the ball is indirectly actuated via bowl motion and gravity.

We investigate two control strategies built on a common simulation and MPC framework:

- 1) **MPC-based tilt-and-roll throwing:** a single reward function drives catching, returning to a nominal pose, spinning the bowl in a horizontal circle, tilting toward the throw direction, and then releasing the ball through a “tilt-and-roll” motion.
- 2) **Phase-based energy-shaping:** the throw is explicitly divided into Spin, Tilt, and Release phases; in each phase an “energy-shaping” reward encourages the bowl to spin at a desired angular speed, tilt to redirect kinetic energy, and then accelerate aggressively at release.

The rest of this report is organized as follows. Section II describes the simulation model and predictive-sampling MPC framework. Section III details the direct MPC controller. Section IV describes the phase-based energy-shaping controller and its additional shaping forces. Section V presents a preliminary obstacle-aware juggling prototype. Section VI presents qualitative results and plots, and Section VII discusses limitations and future work.

II. SYSTEM AND CONTROL FRAMEWORK

A. Simulation Model

We use a MuJoCo XML model of the Unitree G1 humanoid, adapted by removing the lower body, and replacing both hands with rigid bowls attached at the wrists. The XML defines actuators for shoulder, elbow, and wrist joints on each arm, which we treat as the controllable degrees of freedom. A single rigid ball is modeled as a free body with full degrees of freedom (3D position and orientation).

The simulation is configured as follows:

- Gravity: $\mathbf{g} = [0, 0, -9.81]^\top$ m/s².
- Timestep: $\Delta t = 0.002$ s (simulation at 500 Hz).
- Integrator: implicit fast integrator with increased solver iterations.

For numerical stability, we aim to minimize use of resets and dampening on the ball, to ensure that it is kept light and responsive. preliminary exploration, however, showed that some use of these is necessary to maintain simulation realism.

The bowl geometry is modeled as a hollowed hemisphere with an outer radius of 0.075 m as specified in the MuJoCo XML. For the MPC controller, we use an effective radius of 0.071 m to approximate the inner bowl surface where the ball primarily contacts during motion. The energy-shaping controller uses the full outer radius $R_{\text{bowl}} = 0.075$ m to ensure accurate collision geometry in MuJoCo’s contact solver.

B. Analytic Landing Prediction

A simple, but crucial, building block for catching a thrown ball is an analytic prediction of where it will cross the bowl’s optimal catching height. Assuming that during free flight the ball is subject only to gravity, its vertical position is

$$z(t) = z_0 + v_z t - \frac{1}{2} g t^2, \quad (1)$$

where z_0 and v_z are the current vertical position and velocity, and $g > 0$ is the gravitational constant. To predict when the ball reaches a target height z_{target} on the way down, we solve

$$\frac{1}{2} g t^2 - v_z t + (z_0 - z_{\text{target}}) = 0 \quad (2)$$

and choose the largest positive root for which the vertical velocity has become downward. Given this time-of-flight t^* , the predicted landing position is

$$\mathbf{p}_{\text{land}} = \begin{bmatrix} x_0 + v_x t^* \\ y_0 + v_y t^* \end{bmatrix}, \quad (3)$$

with (x_0, y_0) and (v_x, v_y) taken from the ball state at prediction time. This predictor is reused by both controllers to decide where the catching arm should move prior to impact.

C. Sampling-Based MPC

All controllers use the `mujoco_mpc` predictive-sampling planner. At every control step the planner:

- Samples a batch of control sequences over a fixed horizon of 0.3 s.
- Simulates the system forward from the current state with each sequence.
- Evaluates a user-specified reward function on the simulated trajectories.
- Refines the best sequences over several iterations.

We use spline-parameterized control sequences with knot spacing of 0.05 s and a planning step of 0.01 s. For the energy-shaping controller, a typical configuration uses 10 samples per improvement step and 2 improvement iterations. The resulting first control in the best sequence is applied to the active arm, while simple PD feedback keeps the inactive arm held still.

D. Contact Handling

In preliminary exploration, pure MuJoCo contacts produced undesirable behaviors in the ball, such as jerking and unrealistic accelerations. To reduce these, we assume dampened/inelastic collisions throughout the catching phase, implemented by two heuristics:

- **Contact damping:** for all bowl-ball contacts (excluding the throwing phase), the ball’s accelerations are multiplicatively damped. This reduces rebounds and helps the ball settle in the bowl after a catch.
- **Momentum reset on catch:** upon the initial contact of the ball and bowl when a catch is made, a soft reset eliminates 90 % of the ball’s velocities. This effectively “sticks” the ball to the bowl in a manner that avoids unrealistic intersections of bodies in collision.

To reduce numerical errors, these resets occur within the simulation's forward kinematics step— after force calculation and before integration.

III. SIMPLE TILT-AND-ROLL JUGGLING

The first controller was designed around the idea of using MPC to handle the entire juggling cycle with a single reward function:

- 1) Catch the ball in one bowl.
- 2) Return the bowl to a nominal neutral pose.
- 3) Spin the bowl in a horizontal circle, keeping the ball contained.
- 4) Tilt the bowl toward the throw direction to build vertical and horizontal speed.
- 5) Release the ball and move the bowl out of the way.

The same predictive-sampling planner is used both when the arm is waiting to catch and when it is throwing.

A. State Machine

A finite-state machine tracks the logical status of each arm, with each flag holding *True* while satisfying the given conditions:

- *active*: if this arm is currently responsible for the ball.
- *caught*: if the ball is currently inside the bowl.
- *returned*: once the bowl returns near its nominal neutral pose after a catch.
- *throwing*: while this arm is executing a spin and tilt throw.
- *released*: once the ball has been released during the current throw.

B. Catching and Tracking

Before throwing, an arm must first reliably catch a thrown ball, and prepare for throwing. These two steps are discussed as follows.

Firstly, **Tracking mode** moves the bowl at a fixed height (i.e. the horizontal plane) while tracking X-Y alignment with the ball above. The reward function for this penalizes the squared horizontal distance between bowl and ball, as well as the deviation of the bowl's height from the nominal catch height:

$$J_{\text{track}} = 600 \|\mathbf{p}_{\text{bowl},xy} - \mathbf{p}_{\text{ball},xy}\|^2 + 300(z_{\text{bowl}} - z_{\text{catch}})^2 + 1.0 \|\mathbf{q} - \mathbf{q}_{\text{ref}}\|^2 + 0.005 \|\mathbf{u}\|^2, \quad (4)$$

where $\mathbf{p}_{\text{bowl},xy}$ and $\mathbf{p}_{\text{ball},xy}$ are the horizontal positions of the bowl and ball, z_{catch} is the nominal catch height, $\mathbf{q}$ is the joint configuration, and $\mathbf{u}$ is the control input. Upon a MuJoCo object contact between the bowl and ball, a successful 'catch' is assumed.

Next, **Return mode** drives the bowl smoothly back to a reference pose. The cost function penalizes deviation from the target position and orientation, as well as joint velocities and effort:

$$J_{\text{return}} = 150(\Delta x^2 + \Delta y^2) + 100\|R_{\text{bowl}} - R_{\text{start}}\|_F^2 + 10(\Delta z)^2 + 0.5 \sum_i \dot{q}_i^2 + 5.0 \|\mathbf{q} - \mathbf{q}_{\text{ref}}\|^2 + 0.01 \|\mathbf{u}\|^2, \quad (5)$$

where $\Delta x = x_{\text{bowl}} - x_{\text{start}}$, $\Delta y = y_{\text{bowl}} - y_{\text{start}}$, $\Delta z = z_{\text{bowl}} - z_{\text{start}}$ are position deviations, and $\|\cdot\|_F$ denotes the Frobenius norm for rotation matrices. Once the bowl is sufficiently aligned in position and orientation for several consecutive steps, the arm is allowed to begin throwing. This Return mode improves robustness in throws by reproducing predictable positions which are well within the workspace.

This controller design was implemented before the analytically predicted landing point described in Section II was devised. Thus, it directly tracked the ball's motion throughout the arc while balancing speed and error. Later iterations of the model utilized the analytical prediction technique.

C. Tilt-and-Roll Throw Design

During the throwing phase, the bowl's center is driven along a circular trajectory of radius R_{spin} in the horizontal X-Y plane. The nominal trajectory is

$$x(t) = c_x + R_{\text{spin}} \cos(\omega t), \quad (6)$$

$$y(t) = c_y + R_{\text{spin}} \sin(\omega t), \quad (7)$$

$$z(t) = z_{\text{bowl},0}, \quad (8)$$

where $[c_x, c_y]^T$ is the spin center in the horizontal plane, $z_{\text{bowl},0}$ is the initial bowl height, and ω is the spin rate. The reward penalizes deviation from this trajectory and encourages the bowl to tilt toward the desired launch direction as time progresses.

Let $\mathbf{d}_{\text{throw}}$ be the desired launch direction in the horizontal plane (left-to-right or right-to-left). A tilt axis orthogonal to both $\mathbf{d}_{\text{throw}}$ and the world z -axis is used to tilt the bowl by an angle $\theta(t)$ that ramps from 0 to a maximum value θ_{max} over a fixed tilt duration. The reward includes a

Frobenius-norm penalty on the difference between the current bowl rotation matrix and the desired tilted orientation.

Within the bowl, the ball should initially stay near the center and low in the bowl to avoid premature releases. The reward therefore penalizes:

- Horizontal distance of the ball from the bowl center exceeding a fraction of the bowl radius;
- Vertical distance of the ball above the bowl base beyond a phase-dependent bound;
- Vertical ball velocity during early spin (to avoid bouncing); and
- Excessive bowl velocities and joint efforts.

$$J_{\text{throw}} = w_7 \|\mathbf{p}_{\text{bowl}} - \mathbf{p}_{\text{circ}}(t)\|^2 + w_8 \|R_{\text{bowl}} - R_{\text{tilt}}(t)\|_F^2 + w_9 \max(0, \|\mathbf{p}_{\text{ball},xy} - \mathbf{p}_{\text{bowl},xy}\| - \alpha R_{\text{bowl}})^2 + w_{10} \max(0, z_{\text{ball}} - z_{\text{bowl},\text{base}} - h_{\text{max}}(t))^2 + w_{11} \max(0, -\dot{z}_{\text{ball}})^2 - w_{12} (\mathbf{v}_{\text{ball},xy} \cdot \mathbf{d}_{\text{throw}}) - w_{13} \dot{z}_{\text{ball}}. \quad (9)$$

As tilt progresses, these penalties are relaxed and the reward begins to *encourage* horizontal ball velocity aligned with $\mathbf{d}_{\text{throw}}$ and upward vertical velocity, so that the ball is effectively "rolled up the rim" and launched into a ballistic trajectory.

IV. PHASE-BASED ENERGY-SHAPING CONTROLLER

The second controller uses the same sampling-based MPC engine, but emphasizes more explicit shaping of the ball's kinetic energy via a three-phase sequence: *spin*, *tilt*, and *release*. This approach is broadly inspired by classical energy-shaping methods in robotics [5]. The key idea is to spin the bowl and ball to build tangential speed, tilt the bowl to give the tangential velocity a vertical component, and then release while accelerating the arm aggressively to inject additional energy at launch.

A. Throw Phases

For each arm the state machine is extended with a phase variable:

- **Spin**: bowl remains approximately upright at a fixed height, spinning about the wrist to circulate the ball in the bowl and build tangential velocity along the rim.

$$J_{\text{spin}} = w_{r,1} (\max\{0, \|\mathbf{p}_{\text{bowl},xy} - \mathbf{p}_{\text{start},xy}\|_2^2 - R_{\text{spin}}^2\}) + w_{r,2} (\max\{0, \|\mathbf{p}_{\text{ball},xy} - \mathbf{p}_{\text{start},xy}\|_2^2 - R_{\text{spin}}^2\})^2 + w_x \max\{0, x_{\text{start}} - x_{\text{bowl}}\}^2 + w_z (z_{\text{bowl}} - z_{\text{start}})^2 + w_{\text{up}} (1 - \mathbf{z}_{\text{bowl}}^T \mathbf{e}_z)^2 + w_{\text{tilt}} (z_{\text{bowl},x}^2 + z_{\text{bowl},y}^2) + w_{\omega} (\dot{\theta}_{\text{wrist}} - s \omega_{\text{spin}})^2 + w_u \|\mathbf{u}\|_2^2, \quad (10)$$

- **Tilt**: bowl tilts toward the desired throw direction at a fixed tilt angle, redirecting the accumulated tangential velocity into an upward and outward component.

$$\mathbf{z}_{\text{tilt}} = \frac{1}{\|\tilde{\mathbf{z}}_{\text{tilt}}\|_2} \tilde{\mathbf{z}}_{\text{tilt}}, \quad \tilde{\mathbf{z}}_{\text{tilt}} = \begin{bmatrix} d_x \sin \theta_{\text{tilt}} \\ d_y \sin \theta_{\text{tilt}} \\ \cos \theta_{\text{tilt}} \end{bmatrix}, \quad J_{\text{tilt}} = \tilde{w}_{xy} \|\mathbf{p}_{\text{bowl},xy} - \mathbf{p}_{\text{start},xy}\|_2^2 + \tilde{w}_z (z_{\text{bowl}} - z_{\text{start}})^2 + w_{\text{tilt},\text{ori}} \|\mathbf{z}_{\text{bowl}} - \mathbf{z}_{\text{tilt}}\|_2^2 + \tilde{w}_{\omega} (\dot{\theta}_{\text{wrist}} - \alpha s \omega_{\text{spin}})^2 + \tilde{w}_u \|\mathbf{u}\|_2^2, \quad (11)$$

- **Release**: bowl and arm accelerate aggressively in the throw direction to increase ball speed and create clearance between ball and bowl.

$$\mathbf{z}_{\text{rel}} = \frac{1}{\|\tilde{\mathbf{z}}_{\text{rel}}\|_2} \tilde{\mathbf{z}}_{\text{rel}}, \quad \tilde{\mathbf{z}}_{\text{rel}} = \begin{bmatrix} \beta d_x \\ \beta d_y \\ -\gamma \end{bmatrix}, \quad J_{\text{release}} = w_{\text{rel},\text{ori}} \|\mathbf{z}_{\text{bowl}} - \mathbf{z}_{\text{rel}}\|_2^2 - w_{\dot{q}} \sum_i |\dot{q}_i| + w_{\text{rel},u} \|\mathbf{u}\|_2^2, \quad (12)$$

Phase transitions are governed by a combination of timers and a user-triggered signal (a spacebar press in the simulation viewer). In a typical experiment, the spin phase is maintained until the operator judges that sufficient speed has been built; then the spacebar triggers a transition to tilt, which after a fixed number of steps triggers release. The release phase ends when either a distance threshold between ball and bowl is exceeded or a maximum duration is reached, at which point the opposing arm becomes active.

B. Energy-Shaping Reward

In each phase, the reward function is modified to reflect the current goal:

In the **Spin phase**, the bowl center is encouraged to stay within a small radius of its neutral position in the horizontal plane, and at a fixed height. Simultaneously, the bowl's orientation is encouraged to remain upright. A key term in the cost encourages the wrist roll joint to spin at a desired angular velocity ω_{spin} , with separate spin directions depending on whether the left or right arm is throwing. The result is a quasi-periodic orbit where the ball circulates around the bowl interior.

To avoid joint-limit issues, the reward also includes penalties that grow sharply when any joint approaches its range limits. This is implemented by comparing each joint's position against its allowed range and penalizing proximity to the boundaries.

In the **Tilt phase**, the bowl is tilted by a fixed angle θ_{tilt} around an axis orthogonal to the desired throw direction and vertical axis. The reward encourages the bowl's z -axis to align with a desired tilt direction vector

$$\mathbf{z}_{\text{tilt}} = \frac{\begin{bmatrix} d_x \sin \theta_{\text{tilt}} \\ d_y \sin \theta_{\text{tilt}} \\ \cos \theta_{\text{tilt}} \end{bmatrix}}{\left\| \begin{bmatrix} d_x \sin \theta_{\text{tilt}} \\ d_y \sin \theta_{\text{tilt}} \\ \cos \theta_{\text{tilt}} \end{bmatrix} \right\|}, \quad (13)$$

where $[d_x, d_y]^T$ is the horizontal throw direction. Position and height regulation terms are relaxed compared to spin to allow the bowl to move more freely. The desired wrist spin rate is reduced slightly to avoid overspeeding as the bowl tilts.

In the **Release phase**, the reward encourages the bowl's z -axis to align with a vector pointing roughly opposite gravity and along the throw direction, while *negatively* weighting joint velocities to encourage rapid motion. A small control-effort penalty prevents excessively large torques. The intention is to generate a final burst of acceleration as the ball leaves the bowl, analogous to a human "flicking" a racket or hand to add energy at release.

In the **Catch and Tracking phases** When the ball is considered caught, the reward reverts to a neutral return mode similar to the MPC controller: the bowl is driven back toward its nominal pose with penalties on pose error, joint velocities, configuration deviations from a reference, and control effort. When the ball is flying and no throw phase is active, the reward guides the bowl either toward the predicted landing point or directly under the ball in the horizontal plane.

$$J_{\text{catch}} = w_{\text{pos}} (|x_{\text{bowl}} - x_{\text{start}}| + |y_{\text{bowl}} - y_{\text{start}}|) + w_R \|R_{\text{bowl}} - R_{\text{start}}\|_F^2 + w_{z,\text{ret}} |z_{\text{bowl}} - z_{\text{start}}| + w_{\dot{q},\text{ret}} \sum_i \dot{q}_i^2 + w_{q,\text{ret}} \|\mathbf{q} - \mathbf{q}_{\text{ref}}\|_2^2 + w_{u,\text{ret}} \|\mathbf{u}\|_2^2, \quad (14)$$

C. Reward Logging

For the energy-shaping controller we log the instantaneous reward at each timestep, as well as the current phase label (spin, tilt, release, catch, track). A plotting script then visualizes:

- the log-magnitude of the total reward over time, and
- the log-magnitude of the reward restricted to each phase.

This log-scale visualization is particularly useful because the costs can vary across several orders of magnitude between phases: for example, release phases can incur large penalties when the ball leaves the bowl in an undesirable direction, while spin and catch phases tend to produce lower-magnitude costs.

V. PRELIMINARY OBSTACLE-AWARE JUGGLING (PROTOTYPE)

In a separate prototype experiment, we investigated extending the catch, track and throw phase controllers to handle virtual obstacles in the ball's flight corridor. The goal was to use an additional MPC controller for automatically generating launch parameters for a ball trajectory which avoid a spherical obstacle between the arms. Due to time and integration complexity, this prototype was not merged into the main tilt-and-roll or energy-shaping controllers, but it provides a proof-of-concept for obstacle-aware juggling.

We augment the MuJoCo XML by programmatically inserting additional spherical bodies under the `worldbody`. Each obstacle is modeled as a mocap body with a transparent red sphere geom of radius

$$R_{\text{obs}} = 0.04 \text{ m},$$

so its pose can be directly controlled via `data.mocap_pos` while remaining fully collision-active with the ball. For easier visualization of obstacle interactions we also shrink the ball radius to

$$R_{\text{ball}} = 0.01 \text{ m},$$

An *ObstacleManager* class maintains the active obstacle set and is invoked only at discrete events: immediately after a successful catch and return to the neutral pose, and before computing the next throw. Obstacles are either generated at random within the corridor between the two bowls, or, after at least one successful throw, placed along the previous ball trajectory with small random deviations. In this second case, the manager records the ball's last flight path as a sequence of 3D positions and selects one or more points from this path as anchor locations for new obstacles, ensuring they remain within the ball's typical flight region but slightly offset from the exact path.

To generate an obstacle-avoiding path (given a desired start position $\mathbf{p}_{\text{start}}$ at the active bowl and a target position $\mathbf{p}_{\text{target}}$ at the opposite bowl), we treat the launch as a ballistic projectile and search over a family of launch angles. Let r be the horizontal distance and Δh the height difference between start and target:

$$r = \|\mathbf{p}_{\text{target},xy} - \mathbf{p}_{\text{start},xy}\|, \quad \Delta h = p_{\text{target},z} - p_{\text{start},z}.$$

For a candidate elevation angle $\theta \in [65^\circ, 85^\circ]$, the minimal speed v that reaches the target under constant gravity g is given by the standard projectile relation

$$v^2 = \frac{gr^2}{2 \cos^2 \theta (r \tan \theta - \Delta h)}, \quad (15)$$

whenever the denominator is positive. We restrict v to a feasible range (approximately 0.8 m/s to 3.0 m/s) and construct the corresponding 3D launch velocity $\mathbf{v}_0$ by combining the horizontal direction toward the target with the vertical component $v \sin \theta$.

For each candidate (θ, v) we discretize the resulting ballistic trajectory

$$\mathbf{p}(t) = \mathbf{p}_{\text{start}} + \mathbf{v}_0 t + \frac{1}{2} \mathbf{g} t^2, \quad t \in [0, T_{\text{max}}],$$

into a finite set of sample points and compute the minimum distance to all active obstacles. A trajectory that intersects any obstacle (distance below the obstacle radius) is assigned a large negative collision penalty. Otherwise, a scalar score trades off clearance and speed:

$$J(\theta, v) = w_c \min(d_{\text{min}}, d_{\text{max}}) - w_s |v - v_{\text{ref}}| + J_{\text{collision}}, \quad (16)$$

where d_{min} is the minimum obstacle clearance along the path, d_{max} caps the benefit of very large clearances, v_{ref} is a reference speed (approximately 1.3 m/s), and $J_{\text{collision}}$ is a large negative penalty if any collision is predicted. The best-scoring candidate is used to set the ball's free-joint linear velocity at launch, while obstacles remain fixed in space until the next catch-and-return event.

The underlying arm controller remains the planar catching MPC described in Section II, with a reward that encourages the active bowl to move under either the analytically predicted landing point or the current ball position. We also log separate reward components for catching and returning, and visualize their magnitudes on a log scale over time, similarly to Figure 1.

VI. RESULTS

Simulation videos and demonstration files for all experiments are available online at the project demo [repository](#).

All experiments were performed in simulation. The initial configuration places the ball in the air above left bowl, with both arms in a symmetric neutral pose. The ball is first caught and thrown from the left to the right arm. Subsequent throws alternate between arms.

A. Qualitative Behavior

In experimental trials, both controllers were able to generate non trivial throws and catches in simulation, reaching up to 4 consecutive exchanges before failure (defined by loss of control of the ball).

The **direct MPC** controller produced smooth bowl motions that roughly followed the desired spin and tilt pattern and could send the ball in the general direction of the opposite bowl. However, with the limited horizon and modest sample count, it was difficult to consistently build sufficient vertical energy while simultaneously keeping the ball contained; premature releases and low, almost horizontal trajectories were common failure modes.

The phase-based **energy-shaping** controller, once tuned, was more reliable at producing higher, more parabolic trajectories. By explicitly separating spin, tilt, and release and adjusting the reward in each phase, it

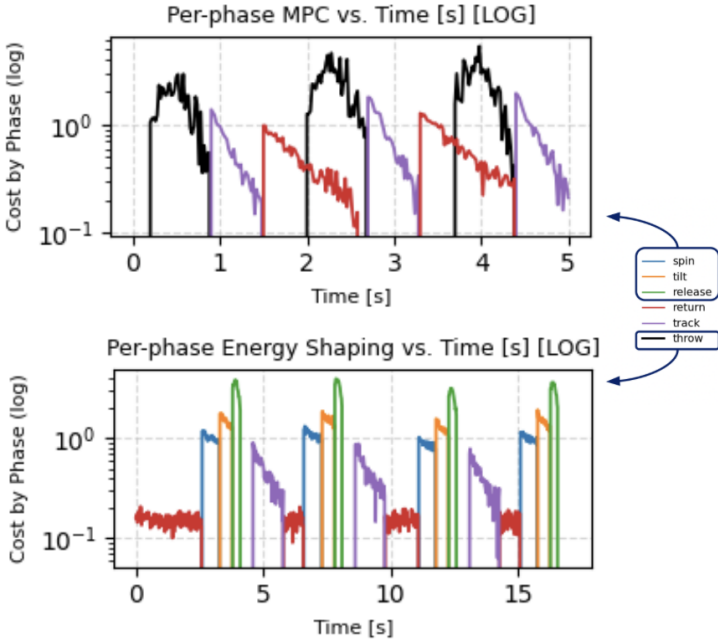


Fig. 1. Representative log-magnitude of the energy shaping and MPC reward vs. time.

was easier to manage conflicting objectives (containment versus energy injection). The additional tuning of these competing objectives led to throws that were visually closer to the desired launch speed and angle.

Catching proved effective for both implementations, with the ability to catch throws that deviated from their intended trajectory. Analytic landing predictions proved more robust than relying solely on direct tracking of the ball’s horizontal position, which required larger and quicker motions. Both catching mechanisms proved to be sensitive to unmodeled disturbances immediately after release, and struggled when making motions which were rapid and near edges of the workspace. These may result from high costs on the bowl’s orientation throughout tracking, resulting in overly disincentivizing the controller from making the necessary accelerations or stretched.

The **obstacle-aware** MPC planning prototype successfully generated throws that curve around obstacles and reached the opposing bowl. It proved robust even when obstacles were placed along previous trajectories-yielding increasingly challenging flight corridors. However, reliability degraded for obstacles placed very close to the nominal path. Moreover, because the launch parameters are computed in a separate high-level module that directly sets the ball velocity, this prototype is not yet consistent with the energy-shaping or tilt-and-roll formulations. A more principled integration would require folding obstacle costs into the MPC reward and allowing the bowl motion itself to realize obstacle-avoiding launches without directly manipulating the ball’s joint velocities.

B. Reward and Phase Analysis

Energy-shaping controller. Figure 1 shows a typical reward log for the energy-shaping controller. The log-magnitude of the reward is plotted over time with separate traces for each phase. Spin and catch phases maintain relatively low cost dominated by gentle pose regulation and spin-rate tracking. Tilt phases show a moderate rise in cost as the bowl departs from the upright configuration. Release phases produce sharp spikes caused by large joint velocities and occasional misaligned releases.

MPC controller. The lower panel of Figure 1 shows the corresponding MPC cost. Tracking phases yield moderate cost magnitudes driven by XY-error and control regularization. Return phases introduce a gradual increase in cost as the optimizer pulls the bowl back to its nominal pose. Tilt and release phases generate broader higher peaks due to strong penalties on bowl orientation, ball containment, and rim approach encoded in the MPC reward. The throw phase exhibits a pronounced peak reflecting the transient combination of rapid arm motion and the incentives for producing the horizontal and vertical velocity components required for a clean launch.

VII. DISCUSSION AND FUTURE WORK

A. Implementation Strengths

Several design choices were particularly helpful:

- **Short-horizon sampling-based MPC:** The sampling-based planner

allowed us to freely modify the reward without changing the underlying optimization algorithm, which made it convenient to iterate on cost terms and phase definitions.

- **Explicit phase structure:** Splitting the throw into phases with distinct objectives (spin, tilt, release) made it easier to tune and reason about the behavior. This was especially true in the energy-shaping controller, where each phase could be associated with a specific energy-transfer purpose.
- **Analytic landing prediction:** Even when not relied on exclusively, analytic landing prediction provided useful information for debugging and helped motivate the design of the tracking reward.
- **Contact heuristics:** The combination of contact damping and momentum reset on catches greatly improved the reproducibility of catches and allowed us to use simpler rewards focused on bowl motion rather than delicate contact timing.
- **Bowl modeling:** MuJoCo collisions do not naively support geometries with concave features. Thus, a custom bowl had been modeled in Blender [7] and segmented into more than 200 individual convex geometries using CoACD [8]. Combined, this model effectively approximated a high-fidelity curvature with collision compatibility.

B. Implementation Challenges

Nevertheless, several aspects were fragile or unsatisfactory:

- **Limited horizon and sampling:** A 0.3s horizon with a modest number of sampled control sequences is marginal for juggling, where flight and preparation times can approach or exceed this scale. This limits the planner’s ability to “see” far enough into the future to coordinate complex catch-and-throw sequences.
- **Cost function non-convexity:** Simulations of the direct MPC controller occasionally resulted in motions that entered local optima in the cost function, leading to faulty or incoherent motions. In the energy-shaping controller we mitigated this partly by simplifying the objectives within each phase and avoiding excessively large quadratic penalties that made the landscape too steep. The use of a 0.3s horizon further aided in escaping local optima.
- **Heavy reliance on heuristics:** Many successful behaviors depended on carefully tuned constants (containment radii, damping factors, joint limit margins, etc.), and small changes could dramatically alter performance. This suggests that a more principled design or learning-based approach to tuning would be beneficial.
- **Sim-to-real considerations:** For a physical Unitree G1, forces must arise from contact and joint torques, so these mechanisms would need to be replaced by robust contact-aware control strategies and more accurate contact modeling.

C. Future Work

Several directions could be explored to extend this work:

- **Longer horizons and better sampling:** Increasing the prediction horizon and using more sophisticated sampling strategies (e.g., CEM, importance sampling, or learned proposal distributions) could improve performance for multi throw and multi ball juggling.
- **Learning cost parameters:** Following prior work on combining policy gradients with MPC [2], one could treat the cost weights as trainable parameters and optimize them via gradient-based methods using rollouts, rather than manual tuning.
- **Vision and state estimation:** The current controller uses perfect state information from the simulator. Moving toward real hardware would require incorporating noisy visual tracking of the ball and robust state estimation.
- **Real hardware experiments:** Ultimately, the goal is to deploy these controllers on an actual Unitree G1 platform. This would require careful system identification, safety constraints, integration with available control interfaces such as `unitree_ros` [6], and possibly domain randomization or other sim-to-real transfer techniques.

VIII. CONCLUSION

We have presented a simulation study of single ball juggling between two bowl shaped end-effectors on a dual arm humanoid robot. Both a direct MPC controller and a phase-based energy-shaping controller were implemented using a common sampling-based planning framework in MuJoCo. The controllers demonstrate the feasibility of using relatively simple, handcrafted rewards and state machines to generate dynamic throwing and catching behaviors in simulation, while also highlighting the difficulty of achieving robust, repeatable juggling in the presence of contact dynamics and limited prediction horizons.

ACKNOWLEDGMENTS

REFERENCES

- [1] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012.
- [2] P. Kulchenko and E. Todorov, “Policy gradient methods with model predictive control applied to the ball-bouncing task,” in *Proc. IEEE Symp. on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, 2011.
- [3] Unitree Robotics, “Unitree G1 Humanoid Robot,” product information, accessed 2025.
- [4] P. Kulchenko, “Robot juggles two ping-pong balls,” blog post, Movement Control Laboratory, Univ. Washington, Seattle, WA, USA, 2011. Available: <https://notebook.kulchenko.com/juggling/robot-juggles-two-ping-pong-balls>. Accessed: 2025.
- [5] R. J. Webster III and B. A. Jones, “Energy-shaping control of soft continuum manipulators with in-plane actuation,” *Int. J. Robot. Res.*, vol. 39, no. 7, pp. 842–858, 2020.
- [6] Unitree Robotics, “unitree_ros: ROS packages for Unitree robots,” GitHub repository. Available: https://github.com/unitreerobotics/unitree_ros. Accessed: 2025.
- [7] Blender Foundation, “Blender – a 3D modelling and rendering package,” 2025. [Online]. Available: <https://www.blender.org>
- [8] W. Wei and Y. Liu, “CoACD: Convex approximate decomposition,” arXiv preprint arXiv:2205.02961, 2022. [Online]. Available: <https://arxiv.org/pdf/2205.02961>